

CSCI 232

232 Algorithms & Data Structures

LECTURE: 12

J. L. Pach

TODAY'S AGENDA

1. **Bubble Sort**: Concept & Intuition
2. **Divide and Conquer**: Breaking Down Large Problems
3. **Introduction to SCRUM**: Agile Workflow Basics
4. **User Stories**: Translating Needs into Actions
5. **Requirements**: Functional vs. Non-Functional
6. **Definition of Done (DoD)**: Ensuring Quality

SOFTWARE

Engineering Approach to Algorithm Design

FROM BUBBLE SORT TO DECOMPOSITION & SCRUM METHODOLOGY

1. BUBBLE SORT: CONCEPT & INTUITION

What is Bubble Sort?

It is a comparison-based sorting algorithm that works by repeatedly stepping through a list, comparing adjacent elements, and swapping them if they are in the wrong order.

Why "Bubble"?

Smaller elements gradually "bubble" to the top of the list, while larger elements sink to the bottom, much like bubbles rising in water.

> *Note: We are focusing purely on the logical concept here—no code yet!*

THE CORE IDEA: VISUALIZED

Imagine a row of numbered items. We compare neighbors and swap them if the left one is larger than the right one.

```
Initial Array: [5] [2] [8] [1] [9]

Pass 1:    [2] [5] [1] [8] [9]  ← largest (9) moved to the end
Pass 2:    [2] [1] [5] [8] [9]  ← second largest (8) in place
Pass 3:    [1] [2] [5] [8] [9]  ← fully sorted!
```

> **Key Observation:** After each full pass, at least one element reaches its final, correct position.

2. DIVIDE AND CONQUER: TASK DECOMPOSITION

What does it mean here?

While "Divide and Conquer" is often associated with recursive algorithms (like Merge Sort), in software engineering, it means **decomposing a large, complex problem into smaller, manageable sub-tasks.**

Instead of saying *"Implement a sorting algorithm"* (overwhelming and vague), we break it down:

1. Create a mechanism to swap two items.
2. Create a loop for a single pass through the array.
3. Create an outer loop to repeat passes until sorted.
4. Add logic to handle ascending vs. descending order.

3. INTRODUCTION TO SCRUM & AGILE METHODOLOGY

What is Agile?

A project management philosophy that emphasizes building software incrementally, validating early, and adapting to changes.

What is SCRUM?

A specific Agile framework where teams work in short, time-boxed iterations called **Sprints** (usually 1-2 weeks).

- **Waterfall (Traditional):**
  - Define everything upfront → Build everything → Test at the very end.
- **Agile (SCRUM):**
  - Build a small piece → Test it → Review → Build the next piece.

CORE SCRUM ARTIFACTS

1. **Product Backlog:**
    i. A prioritized list of all features and tasks needed for the final product.
2. **Sprint Backlog:**
    i. The subset of tasks selected for the current iteration.
3. **Increment:**
    i. The usable, working piece of software delivered at the end of a sprint.

To build our tasks efficiently, we need a standard format to describe them...

4. USER STORIES

A **User Story** is a small, user-focused description of a single feature. It translates vague natural language into a structured, actionable requirement.

The Standard Format:

```
As a [type of user / role],
I want [some goal / capability],
so that [some reason / benefit].
```

Why use this?

It forces us to define **who** needs it, **what** they need, and **why** it brings value, preventing useless or misunderstood features.

CASE STUDY: BUBBLE SORT USER STORIES

Bad Requirement:

"Make the bubble sort work." (Too vague, no measurable outcome)

Good User Stories (Decomposed):

Story 1 (Swap):

As a [sorting algorithm],
I want to [compare two adjacent elements and swap them],
so that [I can reorder misplaced values].

Story 2 (Pass):

As a [sorting algorithm],
I want to [iterate through the array once swapping misplaced pairs],
so that [the largest element reaches the end].```

Story 3 (Direction):

As an [API user],
I want to [specify ascending or descending order],
so that [I can reuse the function for different scenarios].

5. Functional vs. Non-Functional Requirements

Before we write code, we must define the rules.

Functional Requirements (FR):

What the system MUST DO.

- FR-1: The system must sort arrays of integers.
- FR-2: The system must support both ascending and descending order.
- FR-3: The system must handle empty arrays without crashing.

Non-Functional Requirements (NFR):

HOW WELL the system must do it (Quality, Performance).

- NFR-1: Sorting must happen in-place (Space complexity: O(1)).
- NFR-2: The system must compile with zero warnings (`-Wall -Wextra`).
- NFR-3: The code must not contain any memory leaks.

6. DEFINITION OF DONE (DOD)

What is DoD?

A shared checklist of quality standards that a task must meet before it is considered officially "complete."

Writing the code is only 50% of the job. A typical Definition of Done includes:

- [ ] Code compiles without errors or warnings.
- [ ] All Acceptance Criteria in the User Story are met.
- [ ] Edge cases (null pointers, size 0, size 1) are handled.
- [ ] Unit tests are written and passing.
- [ ] No memory leaks exist.
- [ ] Peer code review is completed.

SUMMARY: THE ENGINEERING WORKFLOW

1. **Understand:** Read the natural language requirement.
2. **Decompose:** Divide the large problem into small, logical pieces.
3. **Define:** Write User Stories for each piece (`As a... I want... so that...`).
4. **Specify:** Identify Functional and Non-Functional Requirements.
5. **Implement & Test:** Build incrementally, following Agile principles.
6. **Validate:** Check against the Definition of Done.

> *This transforms algorithm design from blank-page paralysis into a predictable, engineered process.*

THANK

You